

```
In [1]: %matplotlib inline
```

线性模型实验

实验目的

让学生掌握对数几率回归算法的原理和实现，以及如何使用交叉验证法和留一法评估模型的泛化能力。

实验要求

学生需要使用Python语言编写对数几率回归算法，并在给定的数据集上进行训练和测试，比较不同验证方法的错误率，并分析结果。

上机内容1

阅读以下代码，结合搜索引擎，为以下代码添加注释。

注意，请将以下代码中random_state替换为自己学号后四位。

```
In [1]: %reset -f
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
random_state = 48

# 注释?
X, y = make_classification(random_state=random_state)
# 注释?
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=random_state)
# 注释?
clf = LogisticRegression(penalty=None, )
# 注释?
clf.fit(X_train, y_train)
# 注释?
y_pred = clf.predict(X_test)
# 注释?
print("Accuracy:", accuracy_score(y_test, y_pred))
# 注释?
print("Coefficients:", clf.coef_, clf.intercept_)

Accuracy: 0.9
Coefficients: [[ 0.56842889 -0.16119711  0.20418444  0.19461742 -0.20034027  2.293
90375
 0.09981    -0.20904306 -0.31715466 -0.38835544 -0.33073095  0.31083764
 0.11038934  0.04199309 -0.21266371 -0.00514737 -0.2436619  -0.04406371
 0.28743159  0.20571603]] [0.15797737]
```

上机内容2

基于以下代码框架（可以适当修改），编写逻辑回归对应代码，并分析结果

1. 导入相关的库和模块，如numpy和matplotlib。
2. 写出数几率回归算法的表达式，包括线性模型、sigmoid函数、损失函数和梯度下降法。
3. 使用python和numpy实现以上线性模型、sigmoid函数、损失函数和梯度下降法。
4. 加载并预处理数据集，将特征和标签分开，将数据集划分为训练集和测试集。
5. 使用梯度下降法训练对数几率回归模型，并输出最优的参数向量。
6. 使用训练好的模型在测试集上进行预测，并计算准确率和错误率。
7. 使用0.5为阈值，画出混淆矩阵，并绘制PR曲线和ROC曲线、计算AUC值。
8. 使用10折交叉验证法和留一法分别对模型进行评估，并比较两种方法的平均错误率。

注意，请将以下代码中random_state替换为自己学号后四位。

```
In [3]: %reset -f
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, KFold, LeaveOneOut
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
random_state = 52

X, y = make_classification(random_state=random_state)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=random_state)

def linear(w, b, X):
    # z = ?
    raise NotImplementedError
    return z

def sigmoid(z):
    # y_hat = ?
    raise NotImplementedError
    return y_hat

def log_likelihood_loss(y_hat, y):
    # loss = ?
    raise NotImplementedError
    return loss

def gradient_descent(w, b, X, y, eta):
    # w, b = ?
    raise NotImplementedError
    return w, b

class LogisticRegression(object):
    def __init__(self, eta=0.01):
        self.w = None
        self.b = None
        self.eta = eta

    def fit(self, X, y, iterations = 10000):
        self.w = np.random.randn(X.shape[-1])
        self.b = np.random.randn()
        for cnt in range(iterations):
            self.w, self.b = gradient_descent(self.w, self.b, X, y, self.eta)
            if cnt % 1000 == 0:
                print(f'Iteration {cnt}, loss: {log_likelihood_loss(sigmoid(linear
```

```
def predict(self, X):  
    y_test_pred = sigmoid(linear(self.w, self.b, X))  
    return np.where(y_test_pred > 0.5, 1, 0)  
  
# clf = LogisticRegression()  
# clf.fit(X_train, y_train)  
# y_pred = clf.predict(X_test)  
# print("Accuracy:", accuracy_score(y_test, y_pred))  
# print(clf.w, clf.b)
```